Middle East Technical University | Department of Computer Engineering

CENG 334

Introduction to Operating Systems

Spring 2025 - 2026

Homework 3 - File System Merge for EXT2 File System

Due date: 01 06 2026, Monday, 23:59

1 Introduction

In this assignment, you are tasked with implementing a program that will take a base file system image and 2 branches of that base file system with different changes and merge the different changes, handling any merge conflicts, to create a master branch, similar to how git works.

2 ext2

2.1 File System Details

The ext2 file system was introduced for Linux in 1993 and was designed based on Unix file system (UFS) principles, which are covered in your course. Understanding the details of ext2 is crucial for comprehending how the extension will function and for writing your code. Although newer versions of Linux use the ext4 file system, ext3 and ext4 are mainly extensions that add journaling and modern features. The fundamental concepts and data structures of ext2 remain the same.

Structure definitions are provided for your use in the `ext2fs.h` header file and some are also shown below.

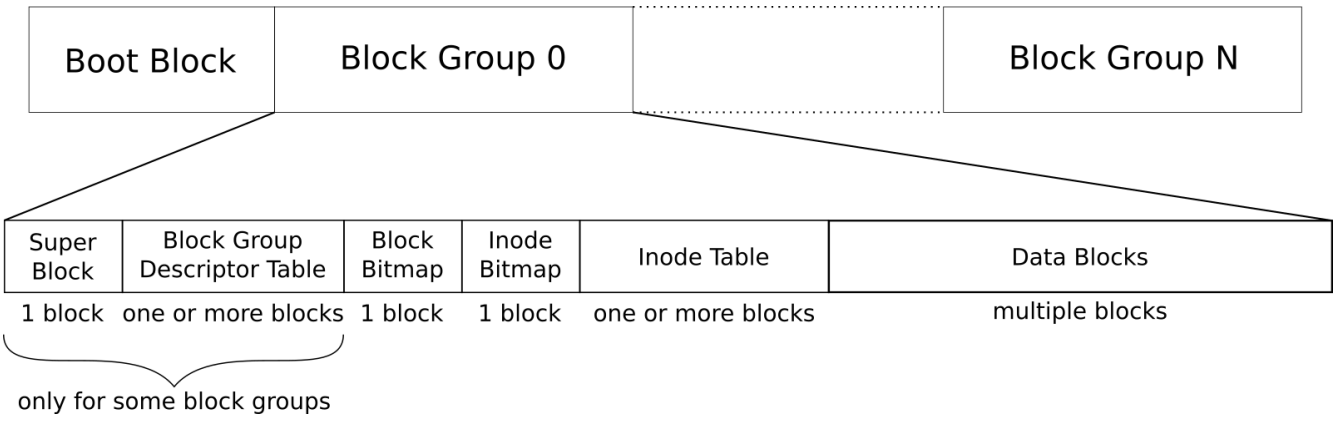

Figure 1: Overall ext2 Layout

An example layout for an ext2 file system is shown in Figure 1. The file system is split into logical block groups to keep file blocks to be closer together on disk. The first 1024 bytes are reserved for boot data (even if unused), followed by the super-block (also 1024 bytes). The super-block contains most of the file system information as a massive structure, part of which you can see below.

Ext2 Super Block Structure

```
struct ext2_super_block {
    uint32_t inode_count; /* Total number of inodes in the fs */
    uint32_t block_count; /* Total number of blocks in the fs */
    uint32_t reserved_block_count; /* Number of blocks reserved for root */
    uint32_t free_block_count; /* Number of free blocks */
    uint32_t free_inode_count; /* Number of free inodes */
    uint32_t first_data_block; /* The first data block number */
    uint32_t log_block_size; /* 2^(10 + this value) gives the block size */
    uint32_t log_fragment_size; /* Same for fragments (we won't use fragments) */
    uint32_t blocks_per_group; /* Number of blocks for each block group */
    uint32_t fragments_per_group; /* Same for fragments */
    uint32_t inodes_per_group; /* Number of inodes for each block group */
    uint32_t mount_time; /* Some less relevant fields */
    uint32_t write_time;
    uint16_t mount_count;
    uint16_t max_mount_count;
    uint16_t magic;
    uint16_t state;
    uint16_t errors;
    uint16_t minor_rev_level;
    uint32_t last_check_time;
    uint32_t check_interval;
    uint32_t creator_os;
    uint32_t rev_level; /* Revision level: 0 or 1 */
    uint16_t default_uid;
    uint16_t default_gid;
    uint32_t first_inode; /* First non-reserved inode in the file system */
    uint16_t inode_size; /* Size of each inode */
    /* More, less relevant fields follow */
};
```

- **Block group descriptor table blocks**: These blocks follow the previously mentioned information.
- **Block group descriptors**: They store information about each block group.
- **Information stored in block group descriptors**: This includes the positions of the bitmaps, inode table, and other details such as the number of free blocks in the block group.
  - **Bitmaps**: These mark allocated block and inode positions in the block group.
  - **Inode table**: It provides space for all the inodes in the block group and is static. The maximum number of inodes is fixed, and unallocated inodes are represented as sequences of zeroes in the table.
  - **Data blocks**: The remaining blocks are allocated for storing data to be used by files.
  - **Backups in some block groups**: Apart from the first block group, some block groups may also contain backups for the superblock and block group descriptor table.

Ext2 Block Group Descriptor Structure

```c
struct ext2_block_group_descriptor {
    uint32_t block_bitmap; /* Block containing the block bitmap */
    uint32_t inode_bitmap; /* Block containing the inode bitmap */
    uint32_t inode_table; /* First block of the inode table */
    uint16_t free_block_count; /* Number of free blocks in the group */
    uint16_t free_inode_count; /* Number of free inodes in the group */
    uint16_t used_dirs_count; /* Number of directories in the group */
    uint16_t pad; /* Padding to 4 byte alignment */
    uint32_t reserved[3]; /* Unused, reserved 12 bytes */
};
```

- **File information stored in inodes**:
  - Mode (type/permissions): Specifies the type of file and its permissions.
  - Owner: Identifies the owner of the file.
  - Size: Indicates the size of the file.
  - Link count: Represents the number of hard links to the file.
  - Timestamps: Stores timestamps related to the file, such as change, modification, and access times.
  - Attributes: Additional attributes associated with the file.
- **Pointers to file data in the inode**:
  - 12 direct blocks: Contains direct pointers to data blocks that store file content.
  - Single indirect block: Contains pointers to other data blocks indirectly, allowing for additional file data storage.
  - Double indirect block: Contains pointers to single indirect blocks, which in turn point to data blocks.
  - Triple indirect block: Contains pointers to double indirect blocks, which further point to single indirect blocks and then to data blocks.
- **Indirect blocks**:
  - Indirect blocks are data blocks filled with pointers.
  - They enable indirect indexing, allowing for multiple data blocks to be accessed through these pointers.

As an example, an ext2 file system with a block size of 4096 will be able to store $4096/4 = 1024$ block numbers in an indirect block. Thus, a single indirect block would be able to index $1024 \cdot 4096 = 4\text{MB}$ of data. A double indirect block would index 1024 indirect blocks, indexing a total of $1024 \cdot 1024 \cdot 4096 = 4\text{GB}$ of data. A triple indirect block would be able to index 4TB of data! The data can have holes, meaning that some intermediate blocks may not be allocated. You can see the inode structure below.[1]

Note that inodes do not contain file names. These are instead contained in directory entries referring to inodes. Thus, it's possible to have entries with different names in different directories referring to the same file (inode). Each of these is called hard links.

[1] Although the maximum file size for a 4KB block ext2 file system would be around 2TB due to the 4-byte limit of the i_blocks field (named block_count_512 in ext2fs.h) in the inode.

Ext2 Inode Structure

```c
struct ext2_inode {
    uint16_t mode; /* Contains filetype and permissions */
    uint16_t uid; /* Owning user id */
    uint32_t size; /* Least significant 32-bits of file size in rev. 1 */
    uint32_t access_time; /* Timestamps (in seconds since 1 Jan 1970) */
    uint32_t change_time;
    uint32_t modification_time;
    uint32_t deletion_time; /* Zero for non-deleted inodes! */
    uint16_t gid; /* Owning group id */
    uint16_t link_count; /* Number of hard links */
    uint32_t block_count_512; /* Number of 512-byte blocks alloc'd to file */
    uint32_t flags; /* Special flags */
    uint32_t reserved; /* 4 reserved bytes */
    uint32_t direct_blocks[12]; /* Direct data blocks */
    uint32_t single_indirect; /* Single indirect block */
    uint32_t double_indirect; /* Double indirect block */
    uint32_t triple_indirect; /* Triple indirect block */
    /* Other, less relevant fields follow */
};
```

Directories are also files and thus have inodes and data blocks. However, data blocks of directories have a special structure. They are filled with directory entry structures forming a singly linked list:

Ext2 Directory Entry Structure

```c
struct ext2_dir_entry {
    uint32_t inode; /* inode number of the file */
    uint16_t length; /* Record length, aligned on 4 bytes */
    uint8_t name_length; /* 255 is the maximum allowed name length */
    uint8_t file_type; /* Not used in rev. 0, file type identifier in rev. 1 */
    char name[]; /* File name. This is called a 'flexible array member' in C. */
};
```

These are records of variable size, essentially 8 bytes plus space for the name, aligned on a 4-byte boundary[2]. The next entry can be reached by adding length bytes to the current entry's offset. The last entry has its length padded to the size of the block so that the next entry corresponds to the offset of the end of the block. Once the end of the block is reached, it's time to move on to the next data block of the directory file. As one last thing, a 0 inode value indicates an entry which should be skipped (can be padding or pre-allocation).

Some important information before finishing up:

- The super block always begins at byte 1024.
- The BGD table is at the beginning of the first unoccupied block after the super block.
- Inode numbering starts at 1. Not zero!
- The first block number depends on the block size and should be read from the superblock.

[2]Why? Remember your computer organization course!

- The first 10 inodes are reserved for various purposes.
- Inode 2 is always the root directory inode.
- Inode 11 usually contains the lost+found directory under root.
- Bitmap and inode table offsets are not fixed. You should not assume anything. Read them from the block group descriptor.
- The last block group can have fewer blocks and inodes than indicated by the *.per_group fields, depending on the total.

The following links contain details about ext2 and should be your go-to references when writing your code:

- ext2 documentation by Dave Poirier: `http://www.nongnu.org/ext2-doc/ext2.html`
- OSDev wiki article on ext2: `https://wiki.osdev.org/Ext2`
- Another, more history-focused article from Dave Poirier: `http://web.mit.edu/tytso/www/linux/ext2intro.html`

Important disclaimer: Some of the references mentions a creation time and not a change time. While the original ext2 documentation mentions a creation time, in order to be POSIX compliant, it is implemented as the change time in the Linux OS. A true change time was not introduced until ext4. More details of how timestamps works can be found in Section 3.2.3.

2.2 Image Creation

To create an ext2 file system image, first, create a zero file via dd. Here's an example run creating a 512KB file (512 1024-byte blocks).

```
$ dd if=/dev/zero of=example.img bs=1024 count=512
```

Then, format the file into a file system image via mke2fs. The following example creates an ext2 file system with 64 inodes and a block size of 2048.

```
$ mke2fs -t ext2 -b 2048 -N 64 example.img
```

You can dump file system details with the dumpe2fs command:

```
$ dumpe2fs example.img
```

Now that you have a file system image, you can mount the image onto a directory. The FUSE-based fuseext2 command allows you to do this in userspace without any superuser privileges (use this on the incks![3]). The below example mounts our example file system in read-write mode:

```
$ mkdir fs-root
$ fuseext2 -o rw+ example.img fs-root
```

[3]Note that **fusermount** does not currently work on the incks without setting group read-execute permissions for the mount directory (chmod g+rx) and group execute permissions (g+x) for other directories on the path to the mount directory (including your home directory). This is not ideal and is being looked into so that it can work with user permissions only. An announcement will follow when a fix happens.

Now, fs-root will be the root of your file system image. You can cd to it, create files and directories, do whatever you want. To unmount once you are done, use fusermount:

```
$ fusermount -u fs-root
```

Make sure to unmount the file system before running programs on the image. On systems where you have superuser privileges, you can use the more standard mount and unmount:

```
$ sudo mount -o rw example.img fs-root
$ sudo umount fs-root
```

You can check the consistency of your file system after modifications with e2fsck. The below example forces the check-in verbose mode and refuses fixes (-n flag), since e2fsck attempts to fix problems by default. This will help you find bugs in your implementation later on.

```
$ e2fsck -fnv example.img
```

3 Implementation

3.1 Display the Merge Status (20 pts)

The ability to see the state of the file system before the merge together with what the merge operation will change is one of the most important and fundamental abilities of any version control program. To achieve this, your program will print whole the directory structure of the file system as a tree (without depth limit). First part of this section only requires you to print existing directory entries, but in the second part you will also print which files are added or modified by which branches. To clarify, you will print a single tree, but you will be graded on the two different criteria separately.

3.1.1 File Hierarchy of the Base Image (10 pts)

Your program will start printing from the root and print the contents of each directory in a depth-first manner. For example, in a file system where "root" is the root directory, "home" and "etc" are its children, and so on, a valid output might look like the following:

```
- root/
-- lost+found/
-- home/
--- user1/
---- file1.txt
---- file2.txt
--- user2/
---- file3.txt
-- etc/
--- config.txt
```

As it can be seen in the example, the name of each directory entry should be preceded by a number of "-" characters equal to the depth level of the entry in the directory hierarchy (root folder having the depth level of 1). The order of entries in a directory does not matter, but you must preserve the hierarchy. This means that for the previous directory hierarchy, the following is also a valid output:

```
- root/
-- lost+found/
-- home/
--- user2/
---- file3.txt
--- user1/
---- file1.txt
---- file2.txt
-- etc/
--- config.txt
```

While the following is not:

```
- root/
-- lost+found/
-- home/
--- user1/
---- file1.txt
---- file2.txt
---- file3.txt
--- user2/
-- etc/
--- config.txt
```

Be sure not to print the "./" or "../" directory entries, as you will lose points, and printing them might cause your code to get stuck in an infinite loop, which might cause you to lose all points.

3.1.2 File Hierarchy of the Merged Image (10 pts)

The first branch that is given to your program is always called Branch A and the second branch is always called Branch B. If we continue from the file system used as an example in the previews section, imagine that

- both Branch A and B edited `file1.txt`,
- Branch B edited `file2.txt`,
- Branch A created a folder named `user3` and a file called `file4.txt` in it, and
- both Branch A and Branch B created a file called `file5.txt` to the folder `user1/`.

Then the directory structure tree with the merge information added to it would look like the following:

```
- root/
-- lost+found/
-- home/
--- user1/
---- file1.txt:MOD:AB
---- file2.txt:MOD:B
---- file5.txt:NEW:AB
--- user2/
---- file3.txt
--- user3/:NEW:A
---- file4.txt:NEW:A
-- etc/
--- config.txt
```

As you can see from the example, modified or newly created files/folders get the tag `MOD` and `NEW` respectively, followed by which branches are modified or created them, all separated by colons (:). The A always comes before the B, regardless of the modification order. Finally, folders can not get the `MOD` tag, only the `NEW` tag. This can be seen in the example as to add the folder `user3` to `home`, the Branch A must must modify not only the data blocks pointed by the inode of `home`, but also the link count as well (Explained more in depth in Section 3.2.3). But there is no `MOD` tag assigned to `home` or any other folder which had files/folders added to it.

3.2 File System Merge (80 pts)

After mapping out the the organization of the file system, you need to merge the changes done in Branch A and Branch B to the base file system image, you will not be modifying the images of Branch A or Branch B, and you wont be creating a new file system image. The changes a branch can do is limited to

- Creating of new directories and files and
- Appending changes to the content of existing files.

Directories and files will not be deleted or moved and existing content of files will not be modified. To check if a branch modified a file will be strictly done by `modification_time` (more on them at Section 3.2.3), meaning if the `modification_time` differs from the base image, the file is considered to be modified by that branch. Creation of directories and files can be seen by changes in inodes, changes in inode bitmaps or changes in directory structures.

IMPORTANT: As a note, if both branches created some directories and/or files, there is no guarantee that they will use distinct and separate inodes and data blocks to do so, meaning you need to be able to handle any inode/data block conflict. As long as all directories, all files and contents of each file exists in the final image, how you handle the conflicts are up to you.

3.2.1 Directory Structure (15 pts)

To count as merged the directory structures of both branches, you need to do the following.

- Allocated inodes for all created directories and files (metadata of these inodes and block pointers of file inodes does not matter **for this step**).
- Have each directory inode's block pointers to point at data blocks containing valid directory structures.
- Have directory entries for each folder and file that exists in the base image, Branch A and Branch B that points to valid inodes located in correct directories.

Meaning, if you are able to create the directory structure you printed at Section 3.1.2 on the actual base file system image, you will complete this section. Your grading will be done by considering how much of the directory structure you modified correctly (ie. if 1 folder and 1 file is added by the branches, and you successfully added the file to the directory structure, you will get half the credit from this section), in a way that can be located by traversing the directory structure (ie. if you created a folder, and its contents perfectly, but didn't create the directory entry that points to it, it will not be counted). As long as inodes and directory entries can be used to traverse the file system, and directory blocks are formatted correctly, no other detail matters **for this step**.

IMPORTANT 1: You need to structure the `length` of directory entries correctly. Especially do not forget that the last directory entry in a data block should span the whole remaining space in that data block.

IMPORTANT 2: When you merge newly created directories to the base image, do not forget to correctly setup their "./" or "../" directory entries.

IMPORTANT 3: When new directory entries are added to an existing directory, the contents of that directory might spill out, meaning you might need to allocate new data blocks to hold all of the directory entries. If this is ever the case, do not forget to point to these data blocks with the original inode.

3.2.2 File Contents (20 pts)

As previously discussed, the existing contents of files will not be modified, but they might be appended to, and contents of the new files need to be copied to the base image as well. There are 4 separate possible cases you need to be mindful of.

1. **File Created by 1 Branch.** If a file is created by a single branch, you just need to allocate an inode to it, copy the whole content of the file from the responsible branch to base image, and have the block pointers of the inode point to those data blocks in the correct order.

2. **File Modified by 1 Branch.** If a file that exists in the base image gets modified by a single branch, you need to copy the difference between the file in the Branch and the file in the base image to data blocks of the base image, and have the existing inode of the file in the base image to correctly point at any new data block you modified.

3. **File Created by 2 Branches.** If a file path that does not exist in the base image exists in both of the branches, first, you need to identify which one of them has the earlier `modification_time`. Then you need to copy the contents of both files, just like you did in the first item, but copy the content of the file that has the earlier `modification_time` first, followed by the later one.

4. **File Modified by 2 Branches.** Similar to item 3, but instead of the whole file content from each file, you need to take the difference between them and the file from the base image, append the content of the difference from the file with earlier `modification_time` first, followed by the difference from the later one.

IMPORTANT 1: If the file you are copying has indirect pointers, make sure that you set them up correctly as well.

IMPORTANT 2: Do not scan the file byte-by-byte to find where the new data begins. Files can contain valid null byte. Instead, to find exactly where the appended data starts, simply read the `size` field of the file's inode from the **base image**. This gives you the exact byte offset where the new content from Branch A or Branch B begins. When both branches modify a file, you will write the appended data from the earlier branch starting at this offset, and the appended data from the later branch immediately following it.

IMPORTANT 3: When allocating new data blocks for a file, you **must** allocate pointers sequentially to minimize disk overhead. You must completely fill all 12 `direct_blocks` before you are allowed to allocate a `single_indirect` block. You must completely fill the single indirect block before allocating a `double_indirect` block, and so on. Do not leave gaps in the direct pointers or skip levels. The semantic grader mathematically calculates your required block delta based on this strict sequential allocation!

3.2.3 Inode Metadata (15 pts)

For every file or directory that is created or modified during the merge operation, you must update its inode metadata to reflect the final merged state.

- `size`: For files, this must be the exact byte count of the final merged file (accounting for exact bytes appended). For directories, this must be a multiple of the file system block size (representing the total size of all allocated data blocks for that directory).

- `block_count_512`: This must reflect the total number of 512-byte sectors allocated to the inode. Do not forget that this count includes data blocks *and* any indirect blocks allocated for indexing!

- `link_count`: For newly created files, this should be set to 1. For directories, you must compute this dynamically: it starts at 2 (the directory itself and its `.` entry), plus 1 for every immediate subdirectory it contains.

  IMPORTANT: If a branch creates a new directory, do not forget to increment the `link_count` of its parent directory!

- `modification_time`, `change_time`, and `access_time`: For all three of these timestamps, take the maximum (latest) value between the base image and the two branches. Conceptually, they mean:

  - `modification_time` (mtime): Updates when the actual contents (data blocks) of the file change.
  - `change_time` (ctime): Updates when the metadata (inode) of the file changes. Because appending data changes the `size` and `block_count_512` fields inside the inode, `ctime` always updates when `mtime` updates.
  - `access_time` (atime): Updates when the file's contents are read.

  As each represent the latest an event occurred, final state should have the latest value for each.

All other metadata fields (such as `mode` and `uid`) should remain identical to their state in the responsible branch, or the base image in case of a dual-modification.

3.2.4 Bitmaps (10 pts)

For every new data block and new inode you allocate during the merge process, you must update the corresponding bitmaps to mark them as in-use (setting their bits to 1). If a block or inode is already in use by the base image, its bit must remain 1.

You must properly locate and update both the `block_bitmap` and `inode_bitmap` blocks for the specific block group from which you allocated the resource.

IMPORTANT: The bits are ordered little-endian within each byte. This means the *least significant bit* (bit 0) of the first byte represents the first block/inode of that group, and the *most significant bit* (bit 7) represents the eighth.

To set the bit for an item at local index k (where k is the 0-based index relative to the start of the block group):

- The byte offset within the bitmap block is $k/8$.
- The bit offset within that specific byte is $k \bmod 8$.
- You can safely set the bit using a bitwise OR operation: `bitmap[byte_offset] |= (1 << bit_offset)`.

3.2.5 Group Descriptor Tables (10 pts)

For each block group descriptor, you need to modify the following fields.

- `free_block_count`: You should decrement it equal to number of data blocks from that block group you allocated for the merge operation.
- `free_inode_count`: You should decrement it equal to number of inodes from that block group's inode table you allocated for the merge operation.
- `used_dirs_count`: Increment it each time you allocate an inode located in the inode table of the block group, that is used for keeping directories (can be queried from the `mode` of the inode).

All other fields should remain the same.

3.2.6 Super Blocks (5 pts)

In the super block, you need to modify the following fields.

- `free_block_count`: You should decrement it equal to number of data blocks you allocated for the merge operation.
- `free_inode_count`: You should decrement it equal to number of inodes you allocated for the merge operation.
- `mount_time`: Get the latest `mount_time` from the base image and both branches, and update this field's value to that.
- `write_time`: Get the latest `write_time` from the base image and both branches, and update this field's value to that.
- `mount_count`: Get the greatest `mount_count` from the base image and both branches, and update this field's value to that.
- `last_check_time`: Get the latest `last_check_time` from the base image and both branches, and update this field's value to that.

All other fields should remain the same.

3.2.7 Backups of GDT & Super Block (5 pts)

In ext2, the super block and the group descriptor table are backed up in some, but not necessarily all, block groups. To properly finish the merge operation, you must update these backups with the final merged versions of your super block and group descriptor table.

- **Locating Backups:** Do not hardcode block group numbers based on version or feature flags. Instead, calculate the layout dynamically. For any given block group, check the `block_bitmap` pointer in its descriptor. If the block bitmap does not begin at the very first block of that block group, calculated from the `blocks_per_group` field of the super block, it is guaranteed that the blocks preceding the bitmap contain the backups.
- **Updating Backups:** Once you identify a block group that contains backups, copy your final merged super block to the first block of that group, and copy your final merged group descriptor table to the block(s) immediately following it.

3.3 Grading

Your assignment will be evaluated using a **Semantic Grader**. We will not be doing a strict byte-by-byte comparison against a "perfect" answer key, because there are multiple valid ways to allocate specific blocks and inodes in ext2. As long as your final merged file system is structurally valid, contains the correct chronological data, and has accurate bookkeeping, it will be marked correct.

The semantic grader evaluates your file system in three independent phases:

1. **Status Printing (20 pts):** Evaluated strictly on your program's standard output matching the required directory tree and diff format.
2. **Structural Merge (50 pts):** The grader will traverse your modified file system using standard ext2 directory rules. It will check if the newly created folders exist, if the file contents are correctly appended, and if the inode metadata (`size`, `modification_time`, `link_count`, etc.) is accurate.
3. **Bookkeeping & Consistency (30 pts):** The grader will mathematically verify your Bitmaps, Group Descriptor Tables, Super Blocks, and Backups based on the exact resources you claimed.

Because there is no single "correct" layout, your bookkeeping is evaluated using an **Expected Delta** and **Allocation Tracing** system. To receive full points for this section, your modified image must pass the following checks:

- **No Destruction:** You must not flip any existing 1 bits back to 0 in the bitmaps. The base image's allocated resources must remain intact.
- **Expected Inode Delta:** The total number of new inodes created by the merge is deterministic. The grader will check your inode bitmaps to ensure exactly that many new bits were flipped to 1.
- **Traced Validity:** The grader will locate the exact new inodes you marked as allocated in the bitmap and verify that they are populated with valid, legitimate merge data.
- **Data Block Verification:** Because directory data block allocation can vary slightly depending on your implementation, the grader will calculate the total blocks required by your newly allocated inodes with a lower limit assigned by the **Expected Data Block Delta** and verify that your data block bitmaps with that, and your GDTs, and Super Blocks with the internal consistency with any modified bitmaps.

4 Specifications

1. Your code must be written in C or C++.
2. Your implementation will be compiled and evaluated on the ineks, so you should make sure that your code works on them.
3. You are supposed to read the file system data structures into memory, modify them, and write them back to the image. Mounting the file system in your code is forbidden; do not run any other executables from your code using things like system() or exec*().
4. Including POSIX ext2/ext3/ext4 libraries (as well as kernel codes and their variations etc.) is not allowed.
5. The `ext2fs.h` header file is provided for your convenience. You are free to include it, modify it, remove it, or do whatever you want with it.
6. Your submission should include every file you used in your code. No files will be supplied to you.
7. We have a zero-tolerance policy against cheating. All the code you submit must be your own work. Sharing code with your friends, using LLM generated code, or using code from the internet or previous years' homework are all considered plagiarism and strictly forbidden.
8. Follow the course page on ODTUClass for possible updates and clarifications.
9. Please ask your questions on ODTUClass instead of sending an email for questions that do not contain code or solutions so that all may benefit.
10. **Late Submission:** Late submission is allowed but with a penalty of $5 \times \text{day} \times \text{day}$ with a maximum late day count of 3 days.

5 Tips and Tricks

- Completing the Section 3.1 will help you know where is everything, and where everything should go, so it is suggested to complete it before you start with the rest of the assignment.

- When copying a file that is modified by a single branch, because the end result of the file will be identical to the file in that branch, instead of taking the difference of the files, you might wanna consider copying the whole file content from the branch, as if you are creating a new file, but with inode and first couple data blocks you will be using already selected for you.
- If a file is created or modified by only a single branch, its final `size` and `block_count_512` will be identical to that branch's version. You can simply copy these metadata fields directly from the branch's inode instead of recalculating them from scratch.
- When calculating your local bitmap index k, pay close attention to where you are starting! Remember that global inode numbers start at 1, not 0. You must correctly convert the global block/inode number to a local, 0-based index for the specific block group before you attempt your division and modulo math.
- `free_block_count` and `free_inode_count` of the super block should match the sum of the fields with the same names from the block group descriptors.
- To determine if a block group has backups, you first need to know exactly where the block group starts. The starting block number of block group N can be calculated as `first_data_block + (N * blocks_per_group)`. Compare this value to the `block_bitmap` field of that group's descriptor to find the offset!

6 Submission

Submission will be done via ODTUClass. Create a gzipped tarball file named hw3.tar.gz that contains all your source code files together with your Makefile. Your submission should not include other files then source files (*.c, *.cpp, *.h, *.hpp, *.inl), and your Makefile, all other files will be removed before grading. Your archive file also should not contain any subfolders. Your code should compile, and your executable should run with the following command sequence:

```
$ tar -xf hw3.tar.gz
$ make all
$ ./mergeext2fs base_img branch_a branch_b
```

If there is a mistake in any of these steps mentioned above, you will lose 10 points.

Compilation & Scope Requirements: Your `Makefile` **must** compile your code using the following compiler flags: `-Werror=unreachable-code -Werror=return-type` (Do not use CMAKE or similar for your compiler setup). Additionally, your submission will be evaluated strictly on the requirements of this assignment. Do not include extraneous functions, dead code, or utilities outside the scope of the file merging requirements. Submissions containing unused functions or out-of-scope logic, you will lose 20 points.